

Lorsque je vous ai parlé des clés étrangères, et que je vous ai donné la syntaxe pour les créer, j'ai omis de vous parler des deux options fort utiles :

- **ON DELETE**, qui permet de déterminer le comportement de MySQL en cas de suppression d'une référence ;
- **ON UPDATE**, qui permet de déterminer le comportement de MySQL en cas de modification d'une référence.

Nous allons maintenant examiner ces options.

Option sur suppression des clés étrangères Petits rappels

La syntaxe

Voici comment on ajoute une clé étrangère à une table déjà existante :

```
ALTER TABLE nom_table
ADD [CONSTRAINT fk_col_ref]          -- On donne un nom à la clé
    (facultatif)
    FOREIGN KEY colonne              -- La colonne sur laquelle on
ajoute la clé
    REFERENCES table_ref(col_ref);    -- La table et la colonne de
référence
```

NB :

Le principe expliqué ici est exactement le même si l'on crée la clé en même temps que la table. La commande **ALTER TABLE** est simplement plus courte, c'est la raison pour laquelle je l'utilise dans mes exemples plutôt que **CREATE TABLE**.

Le principe

Dans notre table **Animal**, nous avons par **exemple** mis une clé étrangère sur la colonne **race_id**, référençant la **colonne id** de la table **Race**. Cela implique que chaque fois qu'une valeur est insérée dans cette colonne (soit en ajoutant une ligne, soit en modifiant une ligne existante), MySQL va vérifier que cette valeur existe bien dans la colonne **id** de la table **Race**.

Aucun animal ne pourra donc avoir un **race_id** qui ne correspond à rien dans notre base.

Suppression d'une référence

Que se passe-t-il si l'on supprime la race des Boxers ? Certains animaux référencent cette espèce dans leur colonne **race_id**. On risque donc d'avoir des données incohérentes. Or, éviter cela est précisément la raison d'être de notre clé étrangère.

Essayons :

```
DELETE FROM Race WHERE nom = 'Boxer';
```

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint
```

Ouf ! Visiblement, **MySQL** vérifie la contrainte de clé étrangère lors d'une suppression aussi, et empêche de supprimer une ligne si elle contient une référence utilisée ailleurs dans la base (ici, l'**id** de la ligne est donc utilisé par certaines lignes de la table **Animal**).

Mais ça veut donc dire que chaque fois qu'on veut supprimer des lignes de la table **Race**, il faut d'abord supprimer toutes les références à ces races. Dans notre base, ça va encore, il n'y a pas énormément de clés étrangères, mais imaginez si l'**id** de **Race** servait de référence à des clés étrangères dans ne serait-ce que cinq ou six tables. Pour supprimer une seule race, il faudrait faire jusqu'à six ou sept requêtes.

C'est donc ici qu'intervient notre option **ON DELETE**, qui permet de changer la manière dont la clé étrangère gère la suppression d'une référence.

Syntaxe :

Voici comment on ajoute cette option à la clé étrangère :

```
ALTER TABLE nom_table
ADD [CONSTRAINT fk_col_ref]
    FOREIGN KEY (colonne)
    REFERENCES table_ref(col_ref)
    ON DELETE {RESTRICT | NO ACTION | SET NULL | CASCADE}; -- <--
Nouvelle option !
```

Il y a donc quatre comportements possibles, que je vais vous détailler tout de suite (bien que leurs noms soient plutôt clairs) :

RESTRICT, **NO ACTION**, **SET NULL** et **CASCADE**.

RESTRICT ou NO ACTION

- **RESTRICT** est le comportement par défaut. Si l'on essaye de supprimer une valeur référencée par une clé étrangère, l'action est avortée et on obtient une erreur. **NO ACTION** a exactement le même effet.

EXEMPLE :

Soit la table **t1** et la table **t2** définies comme ci-dessous :

```
mysql> desc t1;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int(11)   | NO   | PRI | NULL    | auto_increment |
| marchandises | varchar(50) | NO   |     | NULL    |                |
| code       | int(11)   | NO   | UNI | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

Avec ses données :

```
mysql> select * from t1;
+----+-----+-----+
| id | marchandises | code |
+----+-----+-----+
| 1  | tomate       | 1000 |
| 2  | poison       | 1001 |
| 3  | poulet       | 1010 |
| 4  | pomme        | 1011 |
| 5  | mangue       | 1100 |
+----+-----+-----+
5 rows in set (0.00 sec)
```

Puis arrive la table **t2** :

```
mysql> desc t2;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int(11)   | NO   | PRI | NULL    | auto_increment |
| code       | int(11)   | NO   | UNI | NULL    |                |
| quantite   | int(11)   | NO   |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

Avec ses données :

```
mysql> select * from t2;
+----+-----+-----+
| id | code | quantite |
+----+-----+-----+
| 1  | 1000 | 10000    |
| 2  | 1001 | 3000     |
| 3  | 1010 | 60000    |
| 4  | 1011 | 30000    |
| 5  | 1100 | 25000    |
+----+-----+-----+
5 rows in set (0.00 sec)
```

Ajouton une clé étrangère à t2:

```
mysql> alter table t2 add constraint pk foreign key (code) references t1(code);
Query OK, 5 rows affected (1.17 sec)
Enregistrements: 5  Doublons: 0  Avertissements: 0

mysql> select * from t2;
+-----+-----+-----+
| id | code | quantite |
+-----+-----+-----+
| 1 | 1000 | 10000 |
| 2 | 1001 | 3000 |
| 3 | 1010 | 60000 |
| 4 | 1011 | 30000 |
| 5 | 1100 | 25000 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

Et si on essayait d'effectuer une suppression au niveau de **t1** :

```
mysql> delete from t1 where id=2;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`mabase`.`t2`, CONSTRAINT `pk`
FOREIGN KEY (`code`) REFERENCES `t1` (`code`))
mysql>
```

NB :

Cette équivalence de **RESTRICT** et **NO ACTION** est propre à **MySQL**. Dans d'autres SGBD, ces deux options n'auront pas le même effet (**RESTRICT** étant généralement plus strict que **NO ACTION**).

- **SET NULL**

Si on choisit **SET NULL**, alors tout simplement, **NULL** est substitué aux valeurs dont la référence est supprimée.

Pour reprendre notre exemple, en supprimant la race des Boxers, tous les animaux auxquels on a attribué cette race verront la valeur de leur **race_id** passer à **NULL**.

D'ailleurs, ça me semble plutôt intéressant comme comportement dans cette situation ! Nous allons donc modifier notre clé étrangère **fk_race_id** . C'est-à-dire que nous allons supprimer la clé, puis la recréer avec le bon comportement :

Syntaxe :

```
ALTER TABLE Animal DROP FOREIGN KEY fk_race_id;

ALTER TABLE Animal
ADD CONSTRAINT fk_race_id FOREIGN KEY (race_id) REFERENCES Race(id)
ON DELETE SET NULL;
```

Dorénavant, si vous supprimez une race, tous les animaux auxquels vous avez attribué cette race auparavant auront **NULL** comme **race_id** .

Vérifions en supprimant les Boxers, depuis le temps qu'on essaye !

Effectuant cela au niveau de **t1** et **t2**.

Modification de la clé étrangère et suppression des données :

- Suppression de la clé étrangère :

```
mysql> alter table t2 drop foreign key pk;
Query OK, 0 rows affected (0.36 sec)
Enregistrements: 0 Doublons: 0 Avertissements: 0
```

Nous pouvons aussi **supprimer l'index unique** s'il était nécessaire comme ci-dessous :

```
mysql> alter table t2 drop index code;
Query OK, 0 rows affected (0.34 sec)
Enregistrements: 0 Doublons: 0 Avertissements: 0

mysql> desc t2;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int(11)   | NO   | PRI | NULL    | auto_increment |
| code       | int(11)   | NO   |     | NULL    |                |
| quantite   | int(11)   | NO   |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.02 sec)
```

- **Ajoutant les valeurs nulles** à nos deux colonnes qui seront liées (cela est très important pour l'affichage du **null** après suppression)

```
mysql> alter table t1 change code code int(11) null;
Query OK, 0 rows affected (0.77 sec)
Enregistrements: 0 Doublons: 0 Avertissements: 0

mysql> alter table t2 change code code int(11) null;
Query OK, 0 rows affected (0.83 sec)
Enregistrements: 0 Doublons: 0 Avertissements: 0

mysql> desc t1;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id             | int(11)       | NO   | PRI | NULL    | auto_increment |
| marchandises   | varchar(50)   | NO   |     | NULL    |                |
| code           | int(11)       | YES  | UNI | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> desc t2;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int(11)   | NO   | PRI | NULL    | auto_increment |
| code       | int(11)   | YES  | MUL | NULL    |                |
| quantite   | int(11)   | NO   |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

- **Recréant la clé étrangère** avec l'attribut **on delete set null**:

Avant tout, vérifiant les contenus des deux tables :

```
mysql> select * from t1;
+----+-----+-----+
| id | marchandises | code |
+----+-----+-----+
| 1  | tomate       | 1000 |
| 2  | poison       | 1001 |
| 3  | poulet       | 1010 |
| 4  | pomme        | 1011 |
| 5  | mangue       | 1100 |
+----+-----+-----+
5 rows in set (0.00 sec)

mysql> select * from t2;
+----+-----+-----+
| id | code | quantite |
+----+-----+-----+
| 1  | 1000 | 10000    |
| 2  | 1001 | 3000     |
| 3  | 1010 | 60000    |
| 4  | 1011 | 30000    |
| 5  | 1100 | 25000    |
+----+-----+-----+
5 rows in set (0.00 sec)
```

Ajoutant la clé accompagnée de son attribut **on delete set null** :

```
mysql> alter table t2 add constraint fk foreign key(code) references t1(code) on delete set null;
Query OK, 5 rows affected (0.95 sec)
Enregistrements: 5  Doublons: 0  Avertissements: 0
```

- Supprimant maintenant des codes au niveau de la table père c'est-à-dire **t1** :

```
mysql> select * from t1;
+----+-----+-----+
| id | marchandises | code |
+----+-----+-----+
| 1  | tomate       | 1000 |
| 2  | poison       | 1001 |
| 3  | poulet       | 1010 |
| 4  | pomme        | 1011 |
| 5  | mangue       | 1100 |
+----+-----+-----+
5 rows in set (0.00 sec)

mysql> delete from t1 where id=2;
Query OK, 1 row affected (0.10 sec)

mysql> select * from t1;
+----+-----+-----+
| id | marchandises | code |
+----+-----+-----+
| 1  | tomate       | 1000 |
| 3  | poulet       | 1010 |
| 4  | pomme        | 1011 |
| 5  | mangue       | 1100 |
+----+-----+-----+
4 rows in set (0.00 sec)
```

- Vérifiant maintenant les conséquences au niveau de la table fille (**t2**)

```
mysql> select * from t2;
+----+-----+-----+
| id | code | quantite |
+----+-----+-----+
| 1  | 1000 | 10000    |
| 2  | NULL  | 3000     |
| 3  | 1010 | 60000    |
| 4  | 1011 | 30000    |
| 5  | 1100 | 25000    |
+----+-----+-----+
5 rows in set (0.00 sec)
```

NB :

- Nous constatons effectivement que le **NULL** a été bien ajouté au niveau de la colonne fille (code) de la table **t2** à l'endroit qui portait la référence du code qui a été supprimé de la table **t1**.
- Pour la plupart des cas cela n'est pas conseillé pour les faits de cohérence des données.

• CASCADE

Ce comportement est le plus risqué (et le plus violent !). En effet, **cela supprime purement et simplement toutes les lignes qui référençaient la valeur supprimée !**

Donc, si on choisit ce comportement pour la clé étrangère sur la colonne **espece_id** de la table **Animal**, vous supprimez l'espèce "**Perroquet amazone**" et **POUF !**, quatre lignes de votre table **Animal** (les quatre perroquets) sont supprimées en même temps.

Il faut donc être bien sûr de ce que l'on fait si l'on choisit **ON DELETE CASCADE**. Il y a cependant de nombreuses situations dans lesquelles c'est utile. Prenez par exemple un forum sur un site internet. Vous avez une table *Sujet*, et une table *Message*, avec une colonne **sujet_id**. Avec **ON DELETE CASCADE**, il vous suffit de supprimer un sujet pour que tous les messages de ce sujet soient également supprimés. Plutôt pratique non ?

Je le répète : soyez bien sûrs de ce que vous faites ! Je décline toute responsabilité en cas de perte de données causée par un **ON DELETE CASCADE** inconsidérément utilisé !

Option sur modification des clés étrangères

On peut également rencontrer des problèmes de cohérence des données en cas de modification. En effet, si l'on change par exemple l'*id* de la race "**Singapura**", tous les animaux qui ont l'ancien **id** dans leur colonne **race_id** référenceront une ligne qui n'existe plus. Les modifications de références de clés étrangères sont donc soumises aux mêmes restrictions que la suppression.

Exemple : essayons de modifier l'*id* de la race "**Singapura**".

```
UPDATE Race SET id = 3 WHERE nom = 'Singapura';
```

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint
```

L'option permettant de définir le comportement en cas de modification est donc **ON UPDATE {RESTRICT | NO ACTION | SET NULL | CASCADE}**. Les quatre comportements possibles sont exactement les mêmes que pour la suppression.

RESTRICT et **NO ACTION** : empêche la modification si elle casse la contrainte (comportement par défaut).

SET NULL : met **NULL** partout où la valeur modifiée était référencée.

CASCADE : modifie également la valeur là où elle est référencée.

```

-- Suppression de la clé --
-----
ALTER TABLE Animal DROP FOREIGN KEY fk_race_id;

-- Recréation de la clé avec les bonnes options --
-----
ALTER TABLE Animal
ADD CONSTRAINT fk_race_id FOREIGN KEY (race_id) REFERENCES Race(id)
ON DELETE SET NULL -- N'oublions pas de remettre le ON DELETE
!
ON UPDATE CASCADE;

-- Modification de l'id des Singapura --
-----
UPDATE Race SET id = 3 WHERE nom = 'Singapura';

```

Les animaux notés comme étant des "Singapura" ont désormais leur *race_id* à 3. Parfait !

Petite explication à propos de CASCADE

CASCADE signifie que l'événement est répété sur les tables qui référencent la valeur. Pensez à des "réactions en cascade". Ainsi, une suppression provoquera d'autres suppressions, tandis qu'une modification provoquera d'autres... modifications !

Modifions par exemple la clé étrangère sur *Animal.race_id*, avant de modifier l'*id* de la race "Singapura" (jetez d'abord un œil aux données des tables *Race* et *Animal*, afin de voir les différences).

NB

En règle générale (dans 99,99 % des cas), c'est une très mauvaise idée de changer la valeur d'un *id* (ou de votre clé primaire auto-incrémentée, quel que soit le nom que vous lui donnez). En effet, vous risquez des problèmes avec l'auto-incrément, si vous donnez une valeur non encore atteinte par auto-incrémentation par exemple. Soyez bien conscients de ce que vous faites. Je ne l'ai montré ici que pour illustrer le **ON UPDATE**, parce que toutes nos clés étrangères référencent des clés primaires. Mais ce n'est pas le cas partout. Une clé étrangère pourrait parfaitement référencer un simple index, dépourvu de toute auto-incrémentation, auquel cas vous pouvez vous amuser à en changer la valeur autant de fois que vous le voudrez.

APPLICATION

Autre exemple1 : essayons d'en mettre en évidence à travers nos tables **t1** et **t2**.

- Ayant la vue de nos tables :

```

mysql> select * from t1;
+----+-----+-----+
| id | marchandise | code |
+----+-----+-----+
| 1  | tomate      | 1000 |
| 3  | poulet      | 1010 |
| 4  | pomme       | 1011 |
| 5  | mangue      | 1100 |
+----+-----+-----+
4 rows in set (0.25 sec)

mysql> select * from t2;
+----+-----+-----+
| id | code | quantite |
+----+-----+-----+
| 1  | 1000 | 10000    |
| 2  | NULL  | 3000     |
| 3  | 1010 | 60000    |
| 4  | 1011 | 30000    |
| 5  | 1100 | 25000    |
+----+-----+-----+
5 rows in set (0.25 sec)

```


- Vérifiant aussi les indexes pressant à la table t2 :

```
mysql> show indexes from t2;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
t2	0	PRIMARY	1	id	A	5	NULL	NULL		BTREE		
t2	1	fk	1	code	A	5	NULL	NULL	YES	BTREE		

```
2 rows in set (0.00 sec)
```

- Suppression de la **clé étrangère**

```
mysql> alter table t2 drop foreign key fk;
Query OK, 0 rows affected (0.36 sec)
Enregistrements: 0 Doublons: 0 Avertissements: 0
```

- Ajoutant la clé étrangère associée à l'attribut **on delete cascade** :

```
mysql> alter table t2 add constraint fk foreign key (code) references t1(code) on delete cascade;
Query OK, 5 rows affected (1.09 sec)
Enregistrements: 5 Doublons: 0 Avertissements: 0
```

- Effectuant maintenant la suppression d'une ligne au niveau de la table **t1**

```
mysql> delete from t1 where id=3;
Query OK, 1 row affected (0.32 sec)

mysql> select * from t1;
```

id	marchandises	code
1	tomate	1000
4	pomme	1011
5	mangue	1100

```
3 rows in set (0.00 sec)
```

Notant que nous avons eu à supprimer la ligne 3 c'est-à-dire le **1010**, constatons maintenant les conséquences au niveau de la table **t2**.

```
mysql> select * from t2;
```

id	code	quantite
1	1000	10000
2	NULL	3000
4	1011	30000
5	1100	25000

```
4 rows in set (0.00 sec)
```

Le constat est tel qu'on l'attendait c'est-à-dire que toutes les lignes qui contenaient la référence du code **1010** ont été supprimées sans audience juste après la suppression produite au niveau de la table **t1**.

On update cascade

Comme nous avons eu à l'annoncer auparavant, l'attribut **on update** agit plutôt sur la modification au lieu de la suppression des données. En effet, lorsque qu'une information

référéncée se voit être changée sur la table père (**t1** sur notre cas), toutes les autres tables dans la base contenant cette référence sont modifiées automatiquement au niveau des lignes où la référence a été identifiée.

Application :

- Suppression de la précédente et ajout de la nouvelle clé étrangère contenant l'attribut on update cascade

```
mysql> alter table t2 drop foreign key fk;
Query OK, 0 rows affected (0.38 sec)
Enregistrements: 0  Doublons: 0  Avertissements: 0

mysql> alter table t2 add constraint fk foreign key(code) references t1(code) on update cascade;
Query OK, 4 rows affected (26.08 sec)
Enregistrements: 4  Doublons: 0  Avertissements: 0
```

- Modification de la référence

```
mysql> select * from t1;
+----+-----+-----+
| id | marchandises | code |
+----+-----+-----+
| 1  | tomate       | 1000 |
| 4  | pomme       | 1011 |
| 5  | mangue       | 1100 |
+----+-----+-----+
3 rows in set (0.25 sec)

mysql> update t1 set code='1111' where code='1100';
Query OK, 1 row affected (0.11 sec)
Enregistrements correspondants: 1  Modifiés: 1  Warnings: 0

mysql> select * from t1;
+----+-----+-----+
| id | marchandises | code |
+----+-----+-----+
| 1  | tomate       | 1000 |
| 4  | pomme       | 1011 |
| 5  | mangue       | 1111 |
+----+-----+-----+
3 rows in set (0.00 sec)
```

- Vérifiant maintenant au niveau de la table t2

```
mysql> select * from t2;
+----+-----+-----+
| id | code | quantite |
+----+-----+-----+
| 1  | 1000 | 10000    |
| 2  | NULL | 3000     |
| 4  | 1011 | 30000    |
| 5  | 1111 | 25000    |
+----+-----+-----+
4 rows in set (0.00 sec)
```

Comme on peut le constater sur l'image ci-dessus que le changement de la référence s'est fait automatiquement.

NB :

- Pour les modifications, le mieux ici est de laisser le comportement par défaut (**RESTRICT**) pour toutes nos clés étrangères. Les colonnes référencées sont chaque fois des colonnes auto-incrémentées, on ne devrait donc pas modifier leurs valeurs.
- Il s'agit de mon point de vue personnel bien sûr. Si vous pensez que c'est mieux de mettre **ON DELETE CASCADE**, faites-le. On peut certainement trouver des arguments en faveur des deux possibilités.

En résumé

- Lorsque l'on crée (ou modifie) une clé étrangère, on peut lui définir deux options : **ON DELETE**, qui sert en cas de suppression de la référence ; et **ON UPDATE**, qui sert en cas de modification de la référence.
- **RESTRICT** et **NO ACTION** désignent le comportement par défaut : la référence ne peut être ni supprimée, ni modifiée si cela entraîne des données incohérentes vis-à-vis de la clé étrangère.
- **SET NULL** fait en sorte que les données de la clé étrangère ayant perdu leur référence (suite à une modification ou une suppression) soient mises à **NULL**.
- **CASCADE** répercute la modification ou la suppression d'une référence de clé étrangère sur les lignes impactées.

FIN !